Avaliação do Overhead de Desempenho Introduzido por Ferramentas de Observabilidade em Sistemas de Microsserviços

Luiz Matheus Sena Macedo[1], Arthur Anderson Guimarães Lopes[1], Lucas Alves Meneses[1]

Mestrado e Doutorado em Ciência da Computação – MDCC
Universidade Federal do Ceará – UFC, Fortaleza, Ceará, Brasil
senaluiz1307@gmail.com arthur.lopes74@alu.ufc.br
lucasalvesmnss@gmail.com

Abstract. *This paper presents a performance evaluation study focused on quantifying the overhead introduced by observability tools – Prometheus, Grafana, Loki and Jaeger via OpenTelemetry – in a Java Spring Boot microservices environment. The system under evaluation consists of three microservices and an API gateway extracted from the TrainTicket benchmark [Zhou et al., 2021], communicating via HTTP REST through a linear chain, with PostgreSQL persistence and Redis caching. The experimental design follows a* 2×3 *factorial scheme with 30 repetitions per combination (observability on/off* $\times$ *10/50/100 VUs), totaling 180 runs. Statistical significance is assessed at 95% confidence using Shapiro-Wilk normality tests followed by t-tests or Mann-Whitney U tests, following the methodology of Jain [Jain, 1991]. Results with 30 repetitions show that observability overhead is statistically significant at all load levels (*$p < 0{,}001$*): 8.4% at p50/low, 6.1% at p50/medium, and 23.1% at p50/high. Tail latency (p99) increases from 16.9% at low load to 58.5% at high load (+10.2 ms), while the baseline shows no sign of saturation; the mechanism behind the larger overhead at high load was not isolated. Recent works on OpenTelemetry instrumentation overhead [Yang et al., 2024] and trace compression [Chen et al., 2025] are reviewed alongside qualitative observability studies, positioning this work among the first to combine statistical rigor with a recognized microservice benchmark and a full observability stack.*

Resumo. *Este artigo apresenta um estudo de avaliação de desempenho com foco em quantificar o overhead introduzido pelas ferramentas de observabilidade – Prometheus, Grafana, Loki e Jaeger via OpenTelemetry – em um ambiente de microsserviços Java Spring Boot. O sistema sob avaliação é composto por três microsserviços e um gateway de API extraídos do benchmark TrainTicket [Zhou et al., 2021], comunicando-se via HTTP REST em cadeia linear, com persistência em PostgreSQL e cache em Redis. O desenho experimental segue um esquema fatorial* 2×3 *com 30 repetições por combinação (observabilidade ligada/desligada* $\times$ *10/50/100 VUs), totalizando 180 execuções. A significância estatística é avaliada com 95% de confiança usando testes de normalidade Shapiro-Wilk seguidos de testes t ou Mann-Whitney U, conforme a metodologia de Jain [Jain, 1991]. Resultados com 30 repetições mostram que o overhead de observabilidade é estatisticamente significativo em todos os níveis de carga (*$p < 0{,}001$*): 8,4% em p50/carga baixa, 6,1% em p50/carga média e 23,1% em p50/carga alta. A latência de cauda (p99) cresce de 16,9% em carga baixa para 58,5% em carga alta (+10,2 ms), sem sinais de saturação no baseline; o mecanismo por trás do overhead maior em carga alta não foi isolado.*

Palavras-chave: Microsserviços, Observabilidade, Avaliação de Desempenho, Overhead, Prometheus, Jaeger, OpenTelemetry

1. Introdução

A arquitetura de microsserviços consolidou-se como paradigma dominante no desenvolvimento de sistemas distribuídos, impulsionada por atributos como escalabilidade horizontal, implantação independente e isolamento de falhas [Newman, 2015, Thönes, 2015]. Grandes organizações como Netflix, Amazon e Spotify migraram sistemas monolíticos para dezenas ou centenas de serviços autônomos, obtendo agilidade nas entregas e resiliência operacional. No entanto, essa decomposição funcional introduz complexidade substancial: falhas podem propagar-se de forma não linear entre componentes heterogêneos, e a perda de visibilidade sobre o fluxo fim-a-fim das requisições transforma o diagnóstico em uma tarefa dispendiosa e demorada [Soldani et al., 2018].

Nesse cenário, a observabilidade emerge como prática fundamental para a engenharia de confiabilidade de sistemas modernos, sendo definida como a capacidade de inferir o estado interno de um sistema a partir de suas saídas externas [Majors et al., 2022]. Sustentada pelos três pilares de métricas, logs e rastreamento distribuído, a observabilidade permite que equipes de operação detectem anomalias, isolem causas-raiz e antecipem degradações antes que se convertam em indisponibilidade. A stack composta por Prometheus (coleta de métricas), Grafana (visualização), Loki (agregação de logs) e Jaeger (rastreamento distribuído), orquestrada pelo padrão OpenTelemetry, tornou-se a combinação de referência em ambientes cloud-native [Sousa, 2024, Gomes et al., 2022].

Apesar dos benefícios, a instrumentação contínua de telemetria impõe um custo computacional não desprezível sobre os serviços monitorados. Cada coleta de métrica, cada *span* de tracing e cada linha de log exportada consomem ciclos de CPU, memória e largura de banda de rede que, de outra forma, estariam disponíveis para o processamento de requisições de negócio. Estudos recentes começaram a quantificar esse overhead

em nível de *microbenchmarking* [Yang et al., 2024], mas há uma escassez de trabalhos que avaliem, com rigor estatístico e sob carga variável, o impacto da stack completa de observabilidade sobre métricas fim-a-fim como latência, vazão e consumo de recursos. Esta lacuna é particularmente relevante em ecossistemas Java Spring Boot, onde o agente OpenTelemetry opera por instrumentação automática no *bytecode* da JVM.

Este trabalho avalia o overhead de desempenho introduzido pela stack de observabilidade sobre um ambiente composto por três microsserviços Spring Boot comunicando-se em cadeia linear via HTTP REST, um gateway de API, um banco de dados PostgreSQL e um cache Redis – todos extraídos ou inspirados no benchmark TrainTicket [Zhou et al., 2021]. O Microsserviço A (`ts-order-service`) expõe o *endpoint* `GET /api/v1/orderservice/order` e, ao receber uma requisição, realiza uma chamada HTTP ao Microsserviço B (`ts-payment-service`), que por sua vez chama o Microsserviço C (`ts-travel-service`). Este último consulta o PostgreSQL com cache Redis e devolve a resposta que percorre a cadeia de volta ao cliente. O gateway (`ts-gateway-service`) atua como ponto de entrada único, roteando requisições externas para o Microsserviço A. A stack de observabilidade é acoplada via OpenTelemetry Java Agent, Micrometer e Promtail, sem modificação no código fonte das aplicações.

O objetivo central é quantificar, com testes estatísticos a 95% de confiança, a degradação de latência nos percentis p50, p95 e p99, a redução de *throughput* e o consumo adicional de CPU e de memória dos contêineres impostos pela stack de observabilidade, sob três níveis distintos de carga (10, 50 e 100 usuários virtuais). A análise emprega testes estatísticos de aderência e hipótese, seguindo a metodologia de Jain [Jain, 1991]. A Seção 2 revisa os trabalhos relacionados e posiciona este estudo frente ao estado da arte. A Seção 3 detalha a metodologia experimental. A Seção 4 apresenta os resultados. A Seção 5 discute as implicações e ameaças à validade. A Seção 6 conclui o trabalho.

2. Trabalhos Relacionados

A literatura sobre observabilidade em microsserviços pode ser organizada em duas vertentes principais: estudos que avaliam a eficácia das ferramentas para diagnóstico de falhas e trabalhos que investigam o custo computacional da instrumentação. Esta seção revisa representantes de ambas as vertentes, posicionando o presente estudo na interseção entre elas.

Sousa (2024) implementou e avaliou práticas de observabilidade em um microsserviço Spring Boot integrado com Redis e PostgreSQL [Sousa, 2024]. O autor utilizou Prometheus, Grafana, Loki e Jaeger com OpenTelemetry, simulando falhas como interrupção de banco de dados para demonstrar a capacidade diagnóstica das ferramentas. Embora eficaz na detecção de anomalias, o trabalho não realizou análise estatística do overhead imposto pela instrumentação – lacuna central abordada neste estudo.

Gomes et al. (2022) investigaram a observabilidade em arquiteturas monolíticas e de microsserviços *cloud-native* [Gomes et al., 2022]. Utilizando OpenTelemetry integrado com Prometheus, Grafana, Loki e Jaeger, simularam cenários de falha como latência elevada e esgotamento de recursos, comprovando a eficácia das ferramentas na identificação de anomalias. Assim como [Sousa, 2024], o foco foi na capacidade diagnóstica, sem

avaliação quantitativa e estatística do custo de desempenho da instrumentação.

Em uma vertente complementar, **Yang et al. (2024)** utilizaram o *benchmark* MooBench para comparar o overhead de diferentes *frameworks* de observabilidade, incluindo OpenTelemetry e Kieker, em processos JVM nativos e conteinerizados [Yang et al., 2024]. O estudo demonstrou que implementações com *buffer* e compressão reduzem significativamente o custo da instrumentação (até 6,15×), mas restringiu-se a medições de microbenchmarking, sem avaliar o impacto em aplicações reais sob carga variável.

Chen et al. (2025) propuseram o Tracezip, uma técnica de compressão de *spans* implementada diretamente no OpenTelemetry Collector, que explora redundância entre traces para reduzir o volume de dados transmitidos [Chen et al., 2025]. Os experimentos em *benchmarks* de microsserviços demonstraram ganhos expressivos de desempenho na coleta de traces com sobrecarga desprezível. O trabalho, aceito no ISSTA 2025, evidencia que a redução do overhead de tracing é uma linha de pesquisa ativa, mas seu foco está na eficiência da transmissão, e não na quantificação estatística do impacto da stack completa sobre latência, vazão e consumo de recursos.

Tabela 1: Ferramentas de observabilidade adotadas e dimensões de avaliação por trabalho.

Trabalho	Prometheus	Grafana	Loki	Jaeger	OTel	Spring Boot	Overhead	Carga
Sousa (2024)	✓	✓	✓	✓	✓	✓	–	–
Gomes et al. (2022)	✓	✓	✓	✓	✓	–	–	–
Yang et al. (2024)	–	–	–	–	✓	–	✓	–
Chen et al. (2025)	–	–	–	–	✓	–	✓	–
Este estudo	✓	✓	✓	✓	✓	✓	✓	✓

OTel = OpenTelemetry. **Overhead** = avaliação quantitativa do custo da instrumentação. **Carga** = múltiplos níveis de usuários virtuais (10, 50, 100 VUs).

Em síntese, os trabalhos revisados convergem em duas direções: a validação qualitativa da eficácia das ferramentas de observabilidade para diagnóstico de falhas [Sousa, 2024, Gomes et al., 2022] e o desenvolvimento de técnicas para mitigação do overhead de tracing [Yang et al., 2024, Chen et al., 2025]. A Tabela 1 evidencia que, embora Prometheus, Grafana, Loki, Jaeger e OpenTelemetry sejam amplamente adotados, nenhum trabalho anterior combina avaliação quantitativa de overhead com múltiplos níveis de carga sobre a stack completa em Spring Boot. O diferencial deste estudo reside na avaliação **quantitativa e estatisticamente rigorosa** do overhead bruto introduzido pela stack completa sobre métricas de desempenho fim-a-fim, preenchendo uma lacuna identificada na literatura e diferenciando-se por avaliar a stack completa sob múltiplos níveis de carga com rigor estatístico.

3. Metodologia Experimental

3.1. Sistema sob Avaliação

O sistema sob avaliação é composto por três microsserviços Spring Boot, um gateway de API, um banco de dados PostgreSQL e um cache Redis, todos extraídos ou inspirados no benchmark TrainTicket [Zhou et al., 2021] e empacotados em contêineres Docker orquestrados via `docker compose`. A Figura 1 ilustra a arquitetura do sistema.

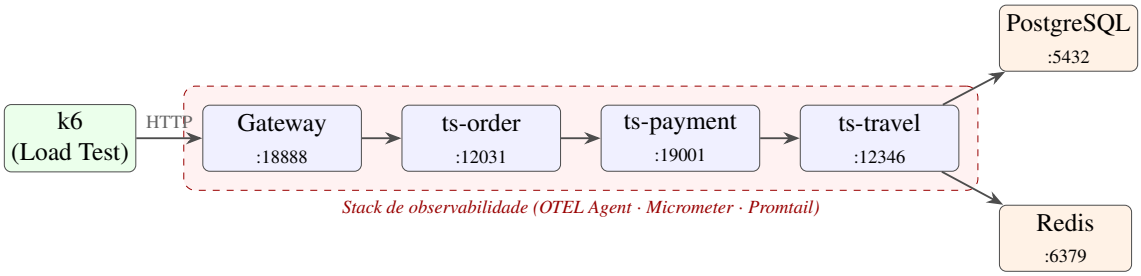


Figura 1: Arquitetura do sistema sob avaliação em cadeia linear. O tráfego entra pelo gateway e percorre a cadeia de serviços até o banco de dados. A pilha tracejada envolve os serviços instrumentados.

O fluxo de requisições segue uma cadeia linear: o gateway (`ts-gateway-service`) roteia requisições para o Microsserviço A (`ts-order-service`), que chama o Microsserviço B (`ts-payment-service`), que chama o Microsserviço C (`ts-travel-service`). Este consulta o PostgreSQL com cache Redis e devolve dados de viagem, cuja resposta percorre a cadeia de volta. Todos os serviços executam sobre JDK 21 com *heap* fixa de 2 GB (`-Xms2g -Xmx2g`) e PostgreSQL compartilhado com três bancos separados: `tsorder`, `tspayment` e `tstravel`.

A stack de observabilidade é acoplada ao ambiente sem modificação no código fonte das aplicações. O OpenTelemetry Java Agent é montado como volume nos contêineres e ativado via `-javaagent`, exportando *spans* para o Jaeger via OpenTelemetry Collector e métricas via Micrometer para o Prometheus. O Promtail coleta os logs gerados pelo Logback e os envia ao Loki. O Grafana consolida as visualizações a partir do Prometheus e do Loki como fontes de dados. A Tabela 2 sumariza a stack e suas funções.

Tabela 2: Stack de observabilidade e respectivas funções.

Ferramenta	**Função**	**Integração**
Prometheus	Coleta de métricas	Micrometer (Actuator)
Grafana	Visualização	Prometheus + Loki
Loki	Agregação de logs	Logback + Promtail
Jaeger	Rastreamento dist.	OTEL Collector
OTEL Collector	Intermediário	Spans→Jaeger, Métricas→Prometheus
OpenTelemetry	Instrumentação	Java Agent (`-javaagent`)

O desenho experimental prevê duas configurações do ambiente: (i) **linha de base** (*baseline*), contendo os três microsserviços, gateway, PostgreSQL e Redis, sem qualquer instrumentação de observabilidade; e (ii) **instrumentado** (*observed*), com a stack de observabilidade completa acoplada. A comparação entre essas duas configurações permite isolar o efeito da observabilidade sobre as métricas de desempenho.

3.2. Workload

A geração de carga é realizada com o **k6**, ferramenta *open-source* de teste de desempenho que executa *scripts* em JavaScript e produz métricas detalhadas de latência, vazão e taxa de erros. A escolha do k6 justifica-se por três fatores: (i) não requer infraestrutura de

cluster, sendo executável em uma única máquina; (ii) oferece suporte nativo a estágios de *warm-up* e rampa de carga via `stages`; e (iii) exporta resultados detalhados em JSON para análise automatizada.

Cada execução do *workload* tem duração total de 5 minutos e 30 segundos, distribuídos em dois estágios: 30 segundos de *warm-up* (rampa até o número alvo de VUs) e 5 minutos em carga constante. O *endpoint* alvo é `GET http://localhost:18888/api/v1/orderservice/order`, com pausa de 100 ms entre requisições consecutivas de um mesmo VU. São utilizados três *scripts* independentes, variando apenas o número de usuários virtuais: 10 VUs (carga baixa), 50 VUs (carga média) e 100 VUs (carga alta).

3.3. Fatores, Parâmetros e Níveis

O experimento adota um desenho fatorial 2×3, com dois fatores controlados e todos os níveis cruzados, totalizando 6 combinações experimentais, conforme a Tabela 3.

Tabela 3: Fatores e níveis do desenho experimental ($2 \times 3 = 6$ combinações).

Fator	**Níveis**	**Descrição**
Configuração de observabilidade	Sem obs.	Microsserviços sem stack
	Com obs.	Stack completa acoplada
Nível de carga (VUs)	10 VUs	Carga baixa (linha de base)
	50 VUs	Carga operacional típica
	100 VUs	Carga de pico / estresse

Os níveis de carga foram escolhidos para cobrir três regimes de operação distintos. A carga baixa (10 VUs) estabelece uma linha de base com mínima contenção de recursos, útil para detectar o custo fixo da instrumentação. A carga média (50 VUs) representa um cenário operacional típico, onde os serviços operam com utilização moderada. A carga alta (100 VUs) representa um cenário de pico. Os resultados (Seção 4) mostram que, mesmo nesse nível, o baseline não chega à saturação: a vazão cresce linearmente com o número de VUs e a latência não aumenta.

As demais variáveis do sistema são mantidas constantes (parâmetros fixos): *heap* máximo da JVM em 2 GB para cada serviço com coletor G1GC (`-XX:+UseG1GC -XX:MaxGCPauseMillis=200`), contêineres sem limite de CPU, rede Docker em modo *bridge* e intervalo de coleta do Prometheus em 15 segundos. Todos os experimentos foram executados em uma única máquina com processador Intel Core i5-12450H (8 núcleos / 12 *threads*), 16 GB de RAM DDR4 e sistema operacional Ubuntu 24.04 LTS (kernel 6.17), com Docker Engine 24.0. A execução em máquina única elimina variáveis de latência de rede entre nós, mas introduz contenção de recursos do hospedeiro, discutida nas ameaças à validade. O número de repetições por combinação é fixado em 30, seguindo a recomendação de Jain [Jain, 1991] para obtenção de intervalo de confiança de 95% com margem de erro controlada, totalizando 180 execuções.

3.4. Métricas de Avaliação

As métricas coletadas abrangem as três dimensões clássicas de avaliação de desempenho – tempo de resposta, vazão e utilização de recursos –, conforme a Tabela 4.

Tabela 4: Métricas de avaliação, ferramentas de coleta e unidades.

Métrica	Coleta	Unid.	O que revela
Latência p50	k6	ms	Custo mediano da instrumentação
Latência p95	k6	ms	Degradação na cauda curta
Latência p99	k6	ms	Degradação na cauda longa
Throughput	k6	req/s	Perda de vazão sob carga
Uso de CPU por serviço	docker stats	%	Custo computacional bruto
Memória por serviço	docker stats	MiB	Memória do contêiner
Erros HTTP	k6	%	Efeito sobre a confiabilidade

As métricas de latência em múltiplos percentis (p50, p95, p99) são preferidas à média aritmética por serem mais robustas a distribuições assimétricas, comuns em sistemas com *garbage collection* e flutuações de rede [Jain, 1991]. O *throughput* é medido em requisições por segundo bem-sucedidas (HTTP 2xx). O uso de CPU é amostrado via `docker stats` ao final de cada execução de teste (uma amostra por execução), sendo reportado como percentual de um núcleo do hospedeiro. A memória de cada serviço é a memória do contêiner coletada via `docker stats`, reportada em MiB; ela inclui o *heap* da JVM, mas não se limita a ele. A taxa de erros HTTP agrega todas as respostas com código diferente de 2xx.

3.5. Justificativa das Escolhas

As escolhas metodológicas deste trabalho são fundamentadas em quatro pilares, conforme preconizado por Jain [Jain, 1991]:

Benchmark. A adoção do TrainTicket como sistema sob avaliação confere validade externa ao estudo. Trata-se de um *benchmark* consolidado, com 41 microsserviços reais e publicações em veículos de alto impacto (IEEE TSE, ICSE, ESEC/FSE) [Zhou et al., 2021]. A extração de três serviços representativos e um gateway – `ts-order-service`, `ts-payment-service`, `ts-travel-service` e `ts-gateway-service` – mantém a identidade do *benchmark* enquanto garante viabilidade operacional em uma única máquina, eliminando variáveis de confusão como latência de rede entre nós e balanceamento de carga.

Stack de observabilidade. A combinação Prometheus + Grafana + Loki + Jaeger + OpenTelemetry é a stack de referência em ambientes *cloud-native*, adotada tanto na indústria quanto nos trabalhos acadêmicos revisados [Sousa, 2024, Gomes et al., 2022, Yang et al., 2024, Chen et al., 2025]. Sua utilização garante que os resultados sejam comparáveis e representativos da prática corrente.

Fatores e níveis. O desenho 2×3 com 30 repetições por célula equilibra poder estatístico e custo experimental. Os três níveis de carga cobrem o espectro de operação realista de um microsserviço, da ociosidade relativa à carga de pico, permitindo caracterizar

se o overhead é predominantemente fixo ou proporcional à carga.

Testes estatísticos. A sequência Shapiro-Wilk → t-test/Mann-Whitney segue o fluxo metodológico padrão em avaliação de desempenho [Jain, 1991]: primeiro verifica-se a aderência à normalidade, depois aplica-se o teste de hipótese adequado (paramétrico ou não-paramétrico), sempre a 95% de confiança. O *script* `analyze.py` implementa essa sequência de forma automatizada utilizando `scipy.stats`.

3.6. Procedimento Experimental

Cada repetição experimental segue um ciclo determinístico de 6 etapas, executado pelo *script* `run-experiments.sh`:

1. **Derrubar** qualquer ambiente residual com `docker compose down -v`;
2. **Subir** o ambiente com `docker compose up -d` (versão baseline ou instrumentada conforme o cenário);
3. **Aguardar** que todos os contêineres atinjam o estado saudável (*healthcheck* com timeout de 180 s) e enviar uma requisição por segundo durante 30 segundos para aquecer a JVM, antes do k6 (cujo estágio de rampa de 30 s é incluído nas métricas);
4. **Executar** o k6 com o *script* correspondente ao nível de carga do cenário, coletando resultados em JSON;
5. **Coletar** métricas de CPU e memória via `docker stats`;
6. **Derrubar** o ambiente com `docker compose down -v` (remoção de volumes para isolamento entre repetições).

O *script* de automação itera sobre os 6 cenários $\times$ 30 repetições (180 execuções), com verificação de progresso e retomada automática em caso de falha, gerando um arquivo JSON por execução e um CSV consolidado com as métricas de recurso.

4. Resultados

Esta seção apresenta os resultados obtidos com 30 repetições por combinação experimental (180 execuções totais), analisados com testes de significância a 95% de confiança.

4.1. Latência

A Tabela 5 sumariza as latências medianas (p50), p95 e p99 para cada combinação experimental, juntamente com o overhead percentual introduzido pela instrumentação de observabilidade.

Tabela 5: Latência (ms) por percentil, configuração e nível de carga. Valores médios de 30 repetições. Overhead entre parênteses.

Carga	Config.	p50	p95	p99
10 VUs	Baseline	6,34	11,58	20,29
	Observed	6,87 (+8,4%)	13,10 (+13,1%)	23,72 (+16,9%)
50 VUs	Baseline	6,15	12,03	20,72
	Observed	6,52 (+6,1%)	13,10 (+8,8%)	22,82 (+10,1%)
100 VUs	Baseline	4,78	11,01	17,49
	Observed	5,88 (+23,1%)	17,23 (+56,6%)	27,73 (+58,5%)

Os resultados mostram que o overhead de observabilidade é estatisticamente significativo em todos os níveis de carga e em todos os percentis ($p < 0{,}001$, teste t de Student ou Mann-Whitney U conforme normalidade). Em carga baixa (10 VUs), o overhead é moderado: 8,4% em p50 e 16,9% em p99. Em carga média (50 VUs), o overhead se mantém controlado: 6,1% em p50 e 10,1% em p99. Em carga alta (100 VUs), o overhead percentual é maior: 23,1% em p50 (+1,10 ms) e 58,5% em p99 (+10,24 ms). Note-se que a latência absoluta do baseline diminui com o aumento da carga (de 6,34 ms a 10 VUs para 4,78 ms a 100 VUs em p50), possivelmente devido a efeitos de cache *warm* e otimizações da JVM sob carga sustentada, o que amplifica o overhead percentual em carga alta. Além disso, o p50 instrumentado em 100 VUs é bimodal: 8 das 30 repetições ficam entre 4,82 e 5,14 ms, próximas do baseline, e as demais entre 5,5 e 6,5 ms. Em p95 e p99 o aumento se mantém também em valores absolutos (+6,2 e +10,2 ms), enquanto o baseline cai. O mecanismo desse aumento não foi isolado neste estudo.

A Tabela 6 apresenta os resultados dos testes de hipótese para cada par configuração $\times$ nível de carga.

Tabela 6: Testes de hipótese para latência (baseline vs. observed). Todos significativos a $\alpha = 0{,}05$.

Métrica	Carga	Teste	p-valor	Overhead
p50	10 VUs	t-test	<0,001	8,4%
p50	50 VUs	Mann-Whitney U	<0,001	6,1%
p50	100 VUs	Mann-Whitney U	<0,001	23,1%
p95	10 VUs	t-test	<0,001	13,1%
p95	50 VUs	t-test	<0,001	8,8%
p95	100 VUs	t-test	<0,001	56,6%
p99	10 VUs	t-test	<0,001	16,9%
p99	50 VUs	t-test	<0,001	10,1%
p99	100 VUs	t-test	<0,001	58,5%

4.2. Throughput

A Tabela 7 apresenta o *throughput* médio em requisições por segundo para cada combinação experimental.

A redução de *throughput* é pequena em todos os níveis: 0,6% e 0,4% em carga baixa e média e 1,8% em carga alta. Ainda assim, ela é estatisticamente detectável nos três

Tabela 7: Throughput médio (req/s) por configuração e nível de carga.

Carga	Baseline	Observed	Redução
10 VUs	88,5	88,0	0,6%
50 VUs	444,8	442,9	0,4%
100 VUs	900,3	884,5	1,8%

níveis (Mann-Whitney U sobre as requisições por execução, $p < 0{,}001$). Como o *workload* é de malha fechada, com pausa de 100 ms por VU, a vazão acompanha o aumento de latência: em 100 VUs, a redução prevista por esse modelo a partir da latência média (1,77%) coincide com a medida (1,75%).

4.3. Latência de Cauda e Distribuição

Em carga alta, o overhead combina deslocamento da mediana (+23%) e alargamento da cauda. A Tabela 8 mostra a porcentagem de requisições que excedem determinados limiares de latência.

Tabela 8: Porcentagem de requisições com latência acima de limiares específicos (média de 30 repetições).

Carga	Config.	>20 ms	>50 ms	>100 ms
10 VUs	Baseline	1,1%	0,0%	0,0%
	Observed	1,7%	0,0%	0,0%
50 VUs	Baseline	1,1%	0,0%	0,0%
	Observed	1,4%	0,0%	0,0%
100 VUs	Baseline	0,7%	0,0%	0,0%
	Observed	3,0%	0,1%	0,0%

Em carga alta com observabilidade, 3,0% das requisições excedem 20 ms, contra apenas 0,7% no baseline, um aumento de cerca de 4,6×. Essa proporção afeta o p99; o p95 instrumentado (17,2 ms) fica abaixo de 20 ms, de modo que seu aumento reflete o deslocamento do corpo da distribuição.

4.4. Decomposição de Latência

A decomposição da latência por fase do processamento (Tabela 9) mostra que 96–99% do aumento do p50 está no tempo de espera do cliente (*waiting*), que engloba o processamento nos serviços (lógica de negócio, instrumentação OTEL e coleta de métricas), os saltos de rede internos da cadeia e a contenção do hospedeiro.

Os tempos de envio (*sending*) e recebimento (*receiving*) são desprezíveis em todas as configurações (<0,1 ms), o que descarta a transferência entre cliente e gateway como causa do overhead.

Tabela 9: Decomposição da latência p50 (ms) por fase de processamento. *Waiting* = espera pela resposta (serviços, rede interna e hospedeiro); *Sending/Receiving* = transferência entre cliente e gateway. Média de 30 repetições.

Carga	Config.	Waiting	Sending	Receiving
10 VUs	Baseline	6,25	0,02	0,05
	Observed	6,78	0,02	0,05
50 VUs	Baseline	6,00	0,02	0,08
	Observed	6,36	0,02	0,09
100 VUs	Baseline	4,64	0,02	0,08
	Observed	5,73	0,01	0,07

4.5. Consumo de Recursos

As Tabelas 10 e 11 apresentam o consumo médio de CPU e memória por serviço em cada cenário. Os valores de memória observados incluem o overhead do agente OTEL e das bibliotecas de instrumentação.

Tabela 10: Consumo médio de CPU (% de um núcleo) por serviço e configuração. Média de 30 repetições, uma amostra por execução.

Carga	Config.	GW	Order	Pay	Travel
10 VUs	Baseline	0,23	0,24	0,22	0,30
	Observed	0,30	0,35	0,33	0,38
50 VUs	Baseline	0,28	0,23	0,27	0,30
	Observed	0,46	0,33	0,58	0,38
100 VUs	Baseline	0,28	0,32	0,32	0,32
	Observed	0,57	0,66	0,70	0,38

Tabela 11: Consumo médio de memória do contêiner (MiB) por serviço e configuração. Média de 30 repetições.

Carga	Config.	GW	Order	Pay	Travel
10 VUs	Baseline	1442	1510	1508	948
	Observed	1507	1591	1589	1410
50 VUs	Baseline	1465	1537	1528	1539
	Observed	1552	1644	1634	1623
100 VUs	Baseline	1490	1558	1553	1548
	Observed	1592	1664	1656	1645

O consumo médio de CPU é maior com instrumentação em todos os serviços e níveis de carga (Tabela 10), mas os valores absolutos são baixos (abaixo de 1% de um núcleo em média) e as médias são sensíveis a picos isolados. Comparando as 30 amostras

por célula com o teste de Mann-Whitney U ($\alpha = 0{,}05$), a diferença é significativa nos quatro serviços em carga alta, mas apenas no *order-service* em carga baixa e apenas no *payment-service* em carga média. Em carga alta, o overhead médio é de +101% no *gateway*, +106% no *order-service*, +115% no *payment-service* e +20% no *travel-service*, que não realiza chamadas downstream. Uma hipótese, não testada aqui por falta de dados por componente, é o custo de criar e exportar *spans* nas chamadas encadeadas.

A memória dos contêineres instrumentados é maior em todos os serviços e níveis de carga (Mann-Whitney U, $p < 0{,}001$ em todas as células), com overhead de 4,6–5,4% em carga baixa, 5,5–7,0% em carga média e 6,2–6,8% em carga alta (Tabela 11). A exceção é o *travel-service* em carga baixa (+49%), cujo consumo no baseline é menor que o dos demais serviços (948 MiB, entre 853 e 1011 MiB por execução, contra cerca de 1,5 GiB nos demais casos); os dados não explicam essa diferença.

Para as execuções instrumentadas com 10 e 50 VUs, o arquivo de métricas de recursos contém dois lotes de 30 execuções; as Tabelas 10 e 11 usam o lote mais recente, coletado na mesma rodada que gerou os dados de latência.

5. Discussão

5.1. Interpretação dos Resultados

Os resultados mostram padrões distintos de overhead conforme o nível de carga:

Carga baixa (10 VUs): O overhead é moderado (8,4% em p50, 16,9% em p99). Entre as fontes prováveis estão o agente OTEL, que cria e exporta *spans*, o Micrometer, que coleta métricas periodicamente, e o Promtail, que processa logs. O valor em p50 fica dentro dos 5–15% reportados por Yang et al. [Yang et al., 2024] para o OTEL agent em microbenchmarks, embora o overhead em p99 (16,9%) exceda essa faixa.

Carga média (50 VUs): O overhead em p50 é o menor entre os três níveis (6,1%), sugerindo que sob regime operacional típico, a instrumentação tem impacto relativamente contido. O overhead em p99 (10,1%) indica que a cauda longa é pouco afetada, possivelmente porque o *batching* de spans do OTEL amortiza o custo por requisição sob carga moderada.

Carga alta (100 VUs): O overhead percentual é o maior dos três níveis, com 23,1% em p50 (+1,10 ms) e 58,5% em p99 (+10,24 ms). Parte do valor em p50 vem da queda da latência do baseline nesse nível, e 8 das 30 execuções instrumentadas praticamente não mostram overhead em p50. A análise de cauda longa (Tabela 8) mostra que 3,0% das requisições excedem 20 ms no cenário observado, contra 0,7% no baseline, um aumento de cerca de 4,6×. A decomposição de latência (Tabela 9) situa o aumento no tempo de espera do cliente, descartando a transferência entre cliente e gateway. O baseline não mostra sinais de saturação nesse nível (vazão linear em relação aos VUs, latência em queda e CPU abaixo de 1% de um núcleo), de modo que o aumento não pode ser atribuído à contenção de recursos com os dados coletados. O aumento de CPU nos serviços que realizam chamadas encadeadas (*order* e *payment*) é compatível com o custo de processar e exportar *spans*, hipótese que exige medição por componente para ser confirmada.

A queda de *throughput* é pequena em todos os níveis de carga (Tabela 7), no

máximo 1,8% em 100 VUs. Com um *workload* de malha fechada e pausa de 100 ms, essa queda é consequência direta do aumento de latência; o experimento não mede a capacidade máxima do sistema, que exigiria carga aberta até a saturação.

5.2. Implicações Práticas

Para equipes que operam microsserviços Spring Boot com observabilidade completa, os resultados sugerem:

- Em ambientes com carga baixa a moderada (até 50 VUs), o overhead da stack OTEL ficou entre 6% e 17% entre p50 e p99, ou de 0,4 a 3,4 ms em valores absolutos;
- Em 100 VUs, o overhead é maior, particularmente na latência de cauda (+58% em p99), o que motiva avaliar estratégias de mitigação como *sampling* de traces, ajuste de *batch size* do OTEL Collector, ou desativação seletiva de spans;
- A latência p99 é a métrica mais sensível ao overhead de observabilidade e deve ser monitorada como indicador primário de degradação;
- O custo de memória da instrumentação (6–7% adicional em carga alta, medido no contêiner) é modesto e estável entre execuções;
- A queda de *throughput* é pequena (no máximo 1,8%), embora estatisticamente detectável; com carga de malha fechada, ela acompanha o aumento de latência.

5.3. Ameaças à Validade

Validade interna: Os resultados são baseados em 30 repetições por combinação, suficientes para detectar as diferenças de latência observadas ($p < 10^{-8}$ em todos os testes); não foi feita análise de poder estatístico. A execução em uma única máquina introduz variáveis de confusão como contenção de recursos do hospedeiro, mitigada pela execução sequencial dos cenários com *teardown* completo entre repetições.

Validade externa: O sistema sob avaliação utiliza três microsserviços em cadeia linear com chamadas síncronas HTTP, o que simplifica topologias reais com dezenas de serviços e comunicação assíncrona. A generalização dos resultados é limitada por três fatores: (i) padrões de *fanout*, em que um serviço chama múltiplos serviços downstream, podem amplificar o overhead de tracing por gerar mais *spans* por requisição; (ii) serviços com acesso intensivo ao banco de dados incorrem em *spans* adicionais de JDBC, cujo overhead não é capturado pela cadeia linear estudada; e (iii) serviços com processamento assíncrono via mensageria (e.g., Kafka, RabbitMQ) podem apresentar padrões de overhead distintos. O uso do Spring Cloud Gateway MVC (síncrono) também pode não representar o comportamento de gateways reativos. A generalização para o benchmark TrainTicket completo, com 41 microsserviços e comunicação assíncrona, requer investigação adicional.

Validade de constructo: A coleta de CPU e memória via `docker stats` é realizada como amostragem instantânea ao final de cada execução. A utilização média de CPU fica abaixo de 1% de um núcleo, mas há picos de até 4,75% (25 das 720 amostras usadas nas tabelas passam de 1%), e uma única amostra por execução não captura esses

picos de forma confiável. Por isso, os resultados de CPU devem ser lidos como indicativos: a diferença é significativa em todos os serviços apenas em carga alta. Séries temporais com intervalos curtos (e.g., 0,5 s) dariam uma medida mais robusta. Quanto à configuração de memória, o *heap* máximo da JVM foi configurado em 2 GB por serviço, valor dentro da faixa típica de produção (1–4 GB). Métricas de GC não foram coletadas. Configurações com *heap* restrito (e.g., 512 MB) poderiam inflacionar artificialmente o overhead da observabilidade, uma vez que a memória adicional consumida pelo agente OTEL e pelas bibliotecas de instrumentação reduziria o espaço disponível para a aplicação, acelerando a frequência de coleta de lixo. Os valores médios de 30 repetições atenuam o impacto de flutuações pontuais.

6. Conclusão

Este trabalho avaliou quantitativamente o overhead de desempenho introduzido pela stack de observabilidade Prometheus + Grafana + Loki + Jaeger via OpenTelemetry em um sistema de três microsserviços Spring Boot em cadeia linear, extraído do benchmark TrainTicket [Zhou et al., 2021]. Com 30 repetições por combinação experimental e testes estatísticos a 95% de confiança, as seguintes conclusões são estabelecidas:

1. O overhead de observabilidade é **estatisticamente significativo** em todos os níveis de carga e percentis de latência ($p < 0{,}001$), confirmando que a instrumentação de telemetria impõe custo mensurável sobre o desempenho;
2. Em **carga baixa e moderada** (10–50 VUs), o overhead é moderado (6–17% entre p50 e p99), com o p50 dentro da faixa reportada na literatura [Yang et al., 2024];
3. Em **carga alta** (100 VUs), o overhead percentual é o maior, sobretudo na cauda: 23% em p50 (+1,1 ms) e 58% em p99 (+10,2 ms), sem sinais de saturação no baseline; o mecanismo desse aumento não foi isolado;
4. A perda de *throughput* é pequena em todos os níveis de carga (no máximo 1,8%) e, com carga de malha fechada, acompanha o aumento de latência;
5. A decomposição de latência situa 96–99% do aumento do p50 no tempo de espera do cliente (serviços, rede interna e hospedeiro), descartando a transferência entre cliente e gateway.

Como trabalho futuro, planeja-se: (i) investigar estratégias de mitigação como *sampling* de traces e ajuste de *batch size*; (ii) avaliar o impacto em topologias com comunicação assíncrona e mais microsserviços; (iii) comparar o overhead entre diferentes agentes de instrumentação (OTEL vs. Skywalking vs. Zipkin).

Referências

[Chen et al., 2025] Chen, Z., Pu, J., and Zheng, Z. (2025). Tracezip: Efficient distributed tracing via trace compression. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. Accepted.

[Gomes et al., 2022] Gomes, I. F., Vidal, V., and Lima, R. (2022). Observabilidade de desempenho em arquiteturas monolíticas e de microsserviços: Uma abordagem cloud-native. *Anais do SBRC*, 40:1–14. Sociedade Brasileira de Computação.

[Jain, 1991] Jain, R. (1991). *The Art of Computer Systems Performance Analysis*. Wiley, New York.

[Majors et al., 2022] Majors, C., Fong-Jones, L., and Miranda, G. (2022). *Observability Engineering: Achieving Production Excellence*. O'Reilly Media, Sebastopol, CA.

[Newman, 2015] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA.

[Soldani et al., 2018] Soldani, J., Tamburri, D. A., and Van Den Heuvel, W.-J. (2018). The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software*, 146:215–232.

[Sousa, 2024] Sousa, R. (2024). Observabilidade em microsserviços: Implementação e avaliação com spring boot. Trabalho de conclusão de curso, Instituto Federal do Ceará.

[Thönes, 2015] Thönes, J. (2015). Microservices. *IEEE Software*, 32(1):116–116.

[Yang et al., 2024] Yang, S., Reichelt, D. G., and Hasselbring, W. (2024). Evaluating the overhead of the performance profiler cloudprofiler with moobench. arXiv:2411.17413 [cs.DC].

[Zhou et al., 2021] Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Li, W., and Ding, D. (2021). Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering*, 47(2):243–263.